

Lab Assignment 7

CMPE 146 Section 1

Professor Bhawandeep Singh Harsh

Andrew Bertch

Robert Trinh

Table of Contents

Exercise 1 – ADC.....	3
Exercise 1.1 – Digitize with software trigger	3
Exercise 1.2 – Measure temperature.....	5
Exercise 2 – PWM.....	9
Exercise 3 – FreeRTOS	9
Exercise 3.1 – Multitask programming to blink LEDs.....	9
Exercise 3.2 - Measurement.....	10
Appendix A – MCU Source Code.....	13
digitize.c	13
temp.c.....	15
pwm.c	18
blink_two_func.c	20
blink_one_func.c	28

Exercise 1 – ADC

In this exercise, we will make use of the temperature sensor and ADC on the MCU. We will use the digitized values produced by the MCU to measure the temperature of the processor on the MCU. This has various uses such as monitoring the chip to ensure it does not overheat, especially when in environments that are warm and using components that are sensitive to temperature.

Exercise 1.1 – Digitize with software trigger

In this exercise, we will digitize the temperature sensor's output voltage and display it on the console. The digital value from the ADC will be displayed below along with the conversion to mV.

CORTEX_MoP: digital
value:1888 0.645313mV

CORTEX_MoP: digital
value:1888 0.645313mV

CORTEX_MoP: digital
value:1888 0.645313mV

CORTEX_MoP: digital
value:1888 0.645313mV

CORTEX_MoP: digital
value:1888 0.645313mV

CORTEX_MoP: digital
value:1888 0.645313mV

CORTEX_MoP: digital
value:1888 0.645313mV

CORTEX_MoP: digital
value:1888 0.645313mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1884 0.643945mV

CORTEX_MoP: digital
value:1884 0.643945mV

CORTEX_MoP: digital
value:1884 0.643945mV

CORTEX_MoP: digital
value:1886 0.644629mV

CORTEX_MoP: digital
value:1886 0.644629mV

CORTEX_MoP: digital
value:1884 0.643945mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1872 0.639844mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1878 0.641895mV

CORTEX_MoP: digital
value:1878 0.641895mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1882 0.643262mV

CORTEX_MoP: digital
value:1878 0.641895mV

CORTEX_MoP: digital
value:1878 0.641895mV

CORTEX_MoP: digital
value:1878 0.641895mV

CORTEX_MoP: digital
value:1878 0.641895mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

CORTEX_MoP: digital
value:1880 0.642578mV

The MCU chip was touched in this order in the console outputs above: no touch, pointer finger touching, no touch, thumb touching.

When finding the quantization error of the measurement, we use the bit resolution of 12-bits and ADC voltage of 1.4V. We can then divide the voltage by the number of values that are possible. This results in a quantization resolution of $1.4/4096 = 0.342$ mV. Dividing the resolution by 2 will find the possible error is 0.171 mV.

Exercise 1.2 – Measure temperature

Now that we have the digital value from the ADC and have converted the value to mV, we can find the temperature that is being measured. The documentation for the MCU provides the necessary equations and values to calculate the temperature being measured. The console outputs that were captured is shown below.

CORTEX_MoP: digital value:1896 0.65 mV 25.23 C/77.42 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1895 0.65 mV 25.42 C/77.76 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1896 0.65 mV 25.23 C/77.42 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1896 0.65 mV 25.23 C/77.42 F

CORTEX_MoP: digital value:1896 0.65 mV 25.23 C/77.42 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1892 0.65 mV 25.99 C/78.79 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1894 0.65 mV 25.61 C/78.10 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1887	0.64 mV	26.94 C/80.49 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1902	0.65 mV	24.09 C/75.37 F
CORTEX_MoP:	digital value:1894	0.65 mV	25.61 C/78.10 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1892	0.65 mV	25.99 C/78.79 F
CORTEX_MoP:	digital value:1892	0.65 mV	25.99 C/78.79 F
CORTEX_MoP:	digital value:1894	0.65 mV	25.61 C/78.10 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1894	0.65 mV	25.61 C/78.10 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F
CORTEX_MoP:	digital value:1888	0.65 mV	26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1884 0.64 mV 27.51 C/81.52 F
CORTEX_MoP: digital value:1884 0.64 mV 27.51 C/81.52 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1884 0.64 mV 27.51 C/81.52 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1884 0.64 mV 27.51 C/81.52 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1886 0.64 mV 27.13 C/80.84 F
CORTEX_MoP: digital value:1884 0.64 mV 27.51 C/81.52 F
CORTEX_MoP: digital value:1884 0.64 mV 27.51 C/81.52 F
CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F
CORTEX_MoP: digital value:1880 0.64 mV 28.27 C/82.89 F
CORTEX_MoP: digital value:1884 0.64 mV 27.51 C/81.52 F
CORTEX_MoP: digital value:1884 0.64 mV 27.51 C/81.52 F
CORTEX_MoP: digital value:1884 0.64 mV 27.51 C/81.52 F

CORTEX_MoP: digital value:1886 0.64 mV 27.13 C/80.84 F

CORTEX_MoP: digital value:1886 0.64 mV 27.13 C/80.84 F

CORTEX_MoP: digital value:1886 0.64 mV 27.13 C/80.84 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

CORTEX_MoP: digital value:1888 0.65 mV 26.75 C/80.15 F

The MCU chip was touched in this order in the console outputs above: no touch, pointer finger touching, no touch, thumb touching. A graph of the temperature in Fahrenheit vs time is shown below. The time can be determined despite not being reported because we have defined a delay of 500 ms.

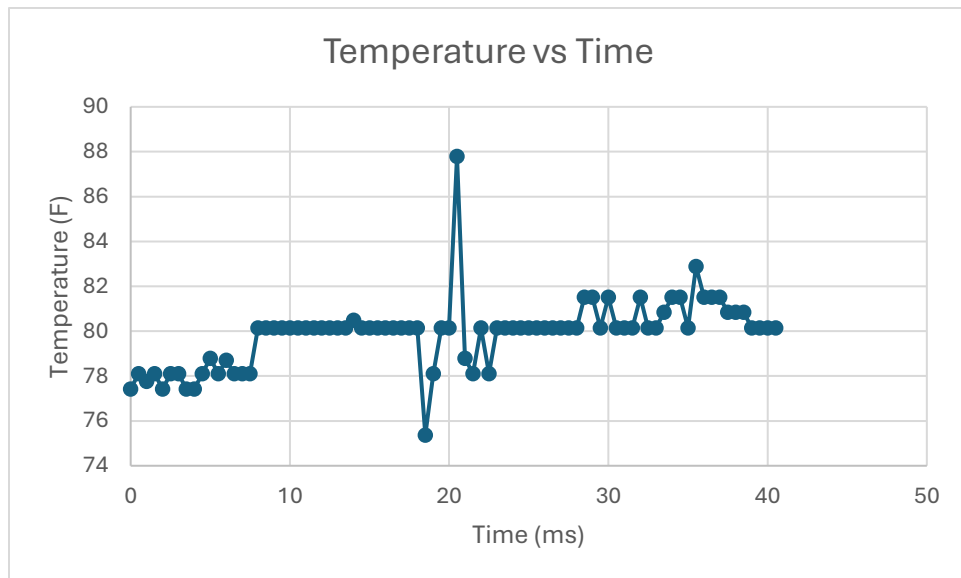


Figure 1.1

The frequency of the oscillator would decrease since RC circuits will have resistances in the components increase. Relevant data on the rise in resistance for each component package can be found in the MSPMoG350x datasheet in section 7.4 Thermal Information. It shows a thermal resistance of ~30-60 °C/W.

Exercise 2 – PWM

In this exercise, we will control the intensity of light emitted from an LED using PWM. We will create our own PWM signal using a ramp time of 2500 ms and a PWM frequency of 20 Hz. We will also have 10 different intensities creating duty cycles of 0-100% with 10% steps between them. The important program parameters for the operation of the LED ramping up and down in 5 seconds is shown below.

CORTEX_MoP: ramp time 2500 modulation frequency 20 modulation resolution 2 num intensities 10

CORTEX_MoP: cycles on: 2

400000 800000 1200000 1600000 2000000 2400000 2800000 3200000 3600000

The modulation frequency of 20 Hz was chosen to be above the 10 Hz specified as the minimum frequency to have easy math for 10 different duty cycles. As a result, the modulation resolution is 2 Hz since $20 \text{ Hz} / 10 \text{ duty cycles} = 2 \text{ Hz/duty cycle}$. A video of the PWM signal ramping up and down the brightness of the LED is included [here](#).

Exercise 3 – FreeRTOS

In this exercise, we will make use of FreeRTOS for the next two exercises. FreeRTOS is a real-time operating system that can be used on the LaunchPad.

Exercise 3.1 – Multitask programming to blink LEDs

In this exercise, we will make use of FreeRTOS to run two different tasks to control different LEDs independently. We will create two different functions that the RTOS will use to make a thread for each task. Within these tasks, the delay value for the red LED uses a delay value of 500000 (500 ms) and the green LED uses a delay value of 2 (2 s).

The code for the blinking is very similar to what we have used previously. We use an infinite loop that sets and clears the pins with delays (in this case, `usleep`) between them. One of the control loops is included on the next page.

```

while(1)
{
    DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_0);
    usleep(500000);
    DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);
    usleep(500000);
}

```

Exercise 3.2 - Measurement

In this exercise, we will create multiple tasks to control an LED but will instead use the same function for each thread. Each thread will be passed the necessary parameters it needs to control different devices on creation of the task and thread to service it. The common task function is shown below.

```

void *LED(void *arg0)
{

    threadArgs *data = (threadArgs*) arg0;
    int ms = (float)(1/data->hz) * 1000;

    printf("IO Mux Pin: %d\n", data->ioMuxPin);
    printf("DL GPIO Pin: %d\n", data->dlGpioPin);

    DL_GPIO_enablePower(data->gpio);

    DL_GPIO_initDigitalOutput(data->ioMuxPin);

    DL_GPIO_initDigitalInput(data->ioMuxPin);

```

```

DL_GPIO_enableOutput(data->gpio, data->dlGpioPin);

DL_GPIO_clearPins(data->gpio, data->dlGpioPin);

while(1)
{
    DL_GPIO_setPins(data->gpio, data->dlGpioPin);
    if(ms < 1000)
        usleep(ms * 1000);
    else
        sleep(ms / 1000);
    DL_GPIO_clearPins(data->gpio, data->dlGpioPin);
    if(ms < 1000)
        usleep(ms * 1000);
    else
        sleep(ms / 1000);
}
}

```

The following code snippet shows the relevant code to create each task in main().

```

threadArgs redArgs;
redArgs.gpio = GPIOA;
redArgs.ioMuxPin = IOMUX_PINCM1;
redArgs.dlGpioPin = DL_GPIO_PIN_0;
redArgs.hz = 2;

threadArgs greenArgs;
greenArgs.gpio = GPIOB;

```

```
greenArgs.ioMuxPin = IOMUX_PINCM58;  
greenArgs.dlGpioPin = DL_GPIO_PIN_27;  
greenArgs.hz = 0.5;
```

```
retcRED = pthread_create(&threadRED, &attrs, LED, &redArgs);  
retcGREEN = pthread_create(&threadGREEN, &attrs, LED, &greenArgs);
```

Each task has a struct made that stores the relevant arguments for the base task function to set itself up to control a specific LED. Once the struct has been populated, a thread is created of the function with the passed arguments. The thread and function will use the data within the arguments struct to populate the DL functions to use a specific LED.

The console outputs of the whole program is shown below.

CORTEX_MoP: IO Mux Pin: 0

CORTEX_MoP: DL GPIO Pin: 1

CORTEX_MoP: IO Mux Pin: 57

CORTEX_MoP: DL GPIO Pin: 134217728

When comparing the two different methods for creating threads controlling LEDs. The second method is more scalable. This is because one function can be reused over and over with arguments passed to it. This means that the code is reusable with the values being easily changed inside main before creating the thread for each task. It also saves on memory since we are reusing code and do not have to create a function for each and every device or task.

Appendix A – MCU Source Code

digitize.c

```
#include "ti_msp_dl_config.h"

#include <stdio.h>

int main(void)
{
    SYSCFG_DL_init();

    static const DL_VREF_ClockConfig gVREFClockConfig = {
        .clockSel = DL_VREF_CLOCK_BUSCLK,
        .divideRatio = DL_VREF_CLOCK_DIVIDE_1,
    };

    static const DL_VREF_Config gVREFConfig = {
        .vrefEnable = DL_VREF_ENABLE_ENABLE,
        .bufConfig = DL_VREF_BUFCONFIG_OUTPUT_1_4V,
        .shModeEnable = DL_VREF_SHMODE_DISABLE,
        .holdCycleCount = DL_VREF_HOLD_MIN,
        .shCycleCount = DL_VREF_SH_MIN,
    };

    DL_VREF_reset(VREF);
    DL_VREF_enablePower(VREF);
    delay_cycles(POWER_STARTUP_DELAY);
    DL_VREF_setClockConfig(VREF, (DL_VREF_ClockConfig *)
    &gVREFClockConfig);
    DL_VREF_configReference(VREF, (DL_VREF_Config *) &gVREFConfig);
```

```

static const DL_ADC12_ClockConfig gADC_INSTClockConfig = {
    .clockSel = DL_ADC12_CLOCK_SYSOSC,
    .divideRatio = DL_ADC12_CLOCK_DIVIDE_4,
    .freqRange = DL_ADC12_CLOCK_FREQ_RANGE_24_TO_32,
};

DL_ADC12_reset(ADC0);
DL_ADC12_enablePower(ADC0);
DL_ADC12_setClockConfig(ADC0, (DL_ADC12_ClockConfig *)
&gADC_INSTClockConfig);
DL_ADC12_initSingleSample(ADC0, DL_ADC12_REPEAT_MODE_ENABLED,
    DL_ADC12_SAMPLING_SOURCE_MANUAL, DL_ADC12_TRIG_SRC_SOFTWARE,
    DL_ADC12_SAMP_CONV_RES_12_BIT,
    DL_ADC12_SAMP_CONV_DATA_FORMAT_UNSIGNED);
DL_ADC12_configConversionMem(ADC0, DL_ADC12_MEM_IDX_0,
    DL_ADC12_INPUT_CHAN_11, DL_ADC12_REFERENCE_VOLTAGE_INTREF,
    DL_ADC12_SAMPLE_TIMER_SOURCE_SCOMP0,
DL_ADC12_AVERAGING_MODE_DISABLED,
    DL_ADC12_BURN_OUT_SOURCE_DISABLED,
DL_ADC12_TRIGGER_MODE_TRIGGER_NEXT,
    DL_ADC12_WINDOWS_COMP_MODE_DISABLED);
DL_ADC12_setPowerDownMode(ADC0, DL_ADC12_POWER_DOWN_MODE_MANUAL);
DL_ADC12_enableConversions(ADC0);

while (1) {
    DL_ADC12_startConversion(ADC0);
    delay_cycles(640);    //delay 2us
    DL_ADC12_stopConversion(ADC0);

```

```

    while (DL_ADC12_isConversionStarted(ADC0)) {
    }

    int value = DL_ADC12_getMemResult(ADC0, DL_ADC12_MEM_IDX_0);
    printf("digital value:%d\t%f mV\n", value, value * (1.4 /
4096));

    delay_cycles(16000000); //delay 500ms
}
}

```

temp.c

```

#include "ti_msp_dl_config.h"
#include <stdio.h>

int main(void)
{
    SYSCFG_DL_init();

    static const DL_VREF_ClockConfig gVREFClockConfig = {
        .clockSel = DL_VREF_CLOCK_BUSCLK,
        .divideRatio = DL_VREF_CLOCK_DIVIDE_1,
    };

    static const DL_VREF_Config gVREFConfig = {
        .vrefEnable = DL_VREF_ENABLE_ENABLE,
        .bufConfig = DL_VREF_BUFCONFIG_OUTPUT_1_4V,
        .shModeEnable = DL_VREF_SHMODE_DISABLE,
        .holdCycleCount = DL_VREF_HOLD_MIN,
    };
}

```

```

        .shCycleCount = DL_VREF_SH_MIN,
    };

    DL_VREF_reset(VREF);
    DL_VREF_enablePower(VREF);
    delay_cycles(POWER_STARTUP_DELAY);
    DL_VREF_setClockConfig(VREF, (DL_VREF_ClockConfig *)
    &gVREFClockConfig);
    DL_VREF_configReference(VREF, (DL_VREF_Config *) &gVREFConfig);

    static const DL_ADC12_ClockConfig gADC_INSTClockConfig = {
        .clockSel = DL_ADC12_CLOCK_SYSOSC,
        .divideRatio = DL_ADC12_CLOCK_DIVIDE_4,
        .freqRange = DL_ADC12_CLOCK_FREQ_RANGE_24_TO_32,
    };

    DL_ADC12_reset(ADC0);
    DL_ADC12_enablePower(ADC0);
    DL_ADC12_setClockConfig(ADC0, (DL_ADC12_ClockConfig *)
    &gADC_INSTClockConfig);
    DL_ADC12_initSingleSample(ADC0, DL_ADC12_REPEAT_MODE_ENABLED,
        DL_ADC12_SAMPLING_SOURCE_MANUAL, DL_ADC12_TRIG_SRC_SOFTWARE,
        DL_ADC12_SAMP_CONV_RES_12_BIT,
        DL_ADC12_SAMP_CONV_DATA_FORMAT_UNSIGNED);
    DL_ADC12_configConversionMem(ADC0, DL_ADC12_MEM_IDX_0,
        DL_ADC12_INPUT_CHAN_11, DL_ADC12_REFERENCE_VOLTAGE_INTREF,
        DL_ADC12_SAMPLE_TIMER_SOURCE_SCOMP0,
    DL_ADC12_AVERAGING_MODE_DISABLED,
        DL_ADC12_BURN_OUT_SOURCE_DISABLED,
    DL_ADC12_TRIGGER_MODE_TRIGGER_NEXT,

```



```

        DL_ADC12_WINDOWS_COMP_MODE_DISABLED);

DL_ADC12_setPowerDownMode(ADC0, DL_ADC12_POWER_DOWN_MODE_MANUAL);

DL_ADC12_enableConversions(ADC0);

while (1) {
    DL_ADC12_startConversion(ADC0);
    delay_cycles(640);    //delay 2us
    DL_ADC12_stopConversion(ADC0);

    while (DL_ADC12_isConversionStarted(ADC0)) {
    }

    int value = DL_ADC12_getMemResult(ADC0, DL_ADC12_MEM_IDX_0);
    float v_sample = (1.4 / 4096) * (value - 0.5);
    float v_trim = (3.3 / 4096) *
(DL_SYSCTL_getTempCalibrationConstant() - 0.5);
    //printf("v_sample %f\tv_trim %f\tsense0 %d\n", v_sample,
v_trim, DL_SYSCTL_getTempCalibrationConstant());
    float t_sample = (1 / -0.0018) * (v_sample - v_trim) + 30;
    printf("digital value:%d\t%.2f mV\t%.2f C/%.2f F\n", value,
value * (1.4 / 4096), t_sample, t_sample * (9.0 / 5) + 32);
    delay_cycles(16000000); //delay 500ms
}
}

```

pwm.c

```
#include "ti_msp_dl_config.h"

#include <stdio.h>


#define RAMP_TIME 2500
#define PWM_FREQ 20
#define NUM_INTENSITIES 10


int main(void)
{
    SYSCFG_DL_init();


    DL_GPIO_enablePower(GPIOA);


    DL_GPIO_initDigitalOutput(IOMUX_PINCM1); // PA0, LED1


    DL_GPIO_enableOutput(GPIOA, DL_GPIO_PIN_0); //PA0, LED1


    DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);


    DL_GPIO_initDigitalOutputFeatures(IOMUX_PINCM1,
    DL_GPIO_INVERSION_ENABLE, DL_GPIO_RESISTOR_NONE,
    DL_GPIO_DRIVE_STRENGTH_LOW, DL_GPIO_HIZ_DISABLE);


    uint32_t intensities[10];
    intensities[0] = 2;
```

```

    printf("ramp time %d modulation frequency %d modulation
resolution%d num intensities%d\n", RAMP_TIME, PWM_FREQ, PWM_FREQ /
NUM_INTENSITIES, NUM_INTENSITIES);

    printf("cycles on: %d", intensities[0]);

    for (int i = 1; i < NUM_INTENSITIES; i++) {
        intensities[i] = ((1.0 / PWM_FREQ) * CPUCLK_FREQ * i / 10) * 5
/ 2;
        printf(" %d ", intensities[i]);
    }

    printf("\n");

    int rise = 1;

    while(1){
        for (int i = 0; i < 10; i += rise) {
            DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_0);
            // printf("on\n");
            delay_cycles(intensities[i] * 5 / 2);
            DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);
            // printf("off\n");
            //printf("%d\n", 1.0 / PWM_FREQ * CPUCLK_FREQ -
intensities[i] * 5 / 2);
            delay_cycles((1.0 / PWM_FREQ * CPUCLK_FREQ) * 5 / 2 -
intensities[i]);
            if ((i == 9 && rise == 1) || (i == 0 && rise == -1)) {
                // printf("here\n");

```

blink_two_func.c

```
* This project provides a POSIX demo using a blinking LED.  
*  
* A POSIX thread controls the blinking LED.  
*  
* This file implements the code that is not demo specific.  
*/
```

```
/* RTOS header files */
```

```

#include <FreeRTOS.h>
#include <task.h>

/* TI includes for driver configuration */
#include "ti_msp_dl_config.h"

void *redLED(void *arg0)
{
    DL_GPIO_enablePower(GPIOA);

    DL_GPIO_initDigitalOutput(IOMUX_PINCM1); // PA0, LED1

    DL_GPIO_initDigitalInput(IOMUX_PINCM1); // PA0, LED1

    DL_GPIO_enableOutput(GPIOA, DL_GPIO_PIN_0); //PA0, LED1

    DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);

    while(1)
    {
        DL_GPIO_setPins(GPIOA, DL_GPIO_PIN_0);
        usleep(500000);
        DL_GPIO_clearPins(GPIOA, DL_GPIO_PIN_0);
        usleep(500000);
    }
}

```

```

void *greenLED(void *arg0)
{
    DL_GPIO_enablePower(GPIOB);

    DL_GPIO_initDigitalOutput(IOMUX_PINCM58); // PB27, green LED in
LED2

    DL_GPIO_initDigitalInput(IOMUX_PINCM58); // PB27, green LED

    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_27); // PB27, green LED

    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);

    while(1)
    {
        DL_GPIO_setPins(GPIOB, DL_GPIO_PIN_27);
        sleep(2);
        DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_27);
        sleep(2);
    }
}

/* Stack size in bytes */
#define THREADSTACKSIZE 1024

/* Set up the hardware ready to run this demo */
static void prvSetupHardware(void);

```

```

int main(void)
{
    pthread_t threadRED, threadGREEN;
    pthread_attr_t attrs;
    struct sched_param priParam;
    int retcRED, retcGREEN;

    /* Initialize the system locks */
#ifdef __ICCARM__
    __iar_Initlocks();
#endif

    /* Prepare the hardware to run this demo. */
    prvSetupHardware();

    /* Initialize the attributes structure with default values */
    pthread_attr_init(&attrs);

    /* Set priority, detach state, and stack size attributes */
    priParam.sched_priority = 1;
    retcRED = pthread_attr_setschedparam(&attrs,
    &priParam);

    retcRED |= pthread_attr_setdetachstate(&attrs,
    PTHREAD_CREATE_DETACHED);

    retcRED |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);

    retcGREEN = pthread_attr_setschedparam(&attrs,
    &priParam);

```

```

    retcGREEN |= pthread_attr_setdetachstate(&attrs,
PTHREAD_CREATE_DETACHED);

    retcGREEN |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);
    if (retcRED != 0) {
        /* failed to set attributes */
        while (1) {
            }
        }
    if (retcGREEN != 0) {
        /* failed to set attributes */
        while (1) {
            }
        }

    retcRED = pthread_create(&threadRED, &attrs, redLED, NULL);
    retcGREEN = pthread_create(&threadGREEN, &attrs, greenLED, NULL);
    if (retcRED != 0) {
        /* pthread_create() failed */
        while (1) {
            }
        }
    if (retcGREEN != 0) {
        /* pthread_create() failed */
        while (1) {
            }
        }
}

```



```

    /* Start the FreeRTOS scheduler */
    vTaskStartScheduler();

    return (0);
}

static void prvSetupHardware(void)
{
    SYSCFG_DL_init();
}

// #if (configSUPPORT_STATIC_ALLOCATION == 1)
// /*
//  * ===== vApplicationGetIdleTaskMemory =====
//  * When static allocation is enabled, the app must provide this
//  callback
//  * function for use by the Idle task.
//  */
// void vApplicationGetIdleTaskMemory(StaticTask_t
// **ppxIdleTaskTCBBuffer,
//      StackType_t **ppxIdleTaskStackBuffer, uint32_t
// *pulIdleTaskStackSize)
// {
//     static StaticTask_t xIdleTaskTCB;
//     static StackType_t
// uxIdleTaskStack[configIDLE_TASK_STACK_DEPTH];

//     *ppxIdleTaskTCBBuffer = &xIdleTaskTCB;

```

```

//      *ppxIdleTaskStackBuffer = uxIdleTaskStack;
//      *pulIdleTaskStackSize    = configIDLE_TASK_STACK_DEPTH;
// }

// #if (configUSE_TIMERS == 1)
// /*
//  * ===== vApplicationGetTimerTaskMemory =====
//  * When static allocation is enabled, and timers are used, the app
//  must provide
//  * this callback function for use by the Timer Service task.
//  */
// void vApplicationGetTimerTaskMemory(StaticTask_t
**ppxTimerTaskTCBBuffer,
//      StackType_t **ppxTimerTaskStackBuffer, uint32_t
*pulTimerTaskStackSize)
// {
//      static StaticTask_t xTimerTaskTCB;
//      static StackType_t
uxTimerTaskStack[configTIMER_TASK_STACK_DEPTH];

//      *ppxTimerTaskTCBBuffer    = &xTimerTaskTCB;
//      *ppxTimerTaskStackBuffer = uxTimerTaskStack;
//      *pulTimerTaskStackSize    = configTIMER_TASK_STACK_DEPTH;
// }
// #endif

// #endif

```

```

#if (configCHECK_FOR_STACK_OVERFLOW)
/*
    * ===== vApplicationStackOverflowHook =====
    * When stack overflow checking is enabled the application must
provide a
    * stack overflow hook function. This default hook function is
declared as
    * weak, and will be used by default, unless the application
specifically
    * provides its own hook function.
*/
#if defined(__IAR_SYSTEMS_ICC__)
__weak void vApplicationStackOverflowHook(
    TaskHandle_t pxTask, char *pcTaskName)
#elif (defined(__TI_COMPILER_VERSION__))
#pragma WEAK(vApplicationStackOverflowHook)
void vApplicationStackOverflowHook(TaskHandle_t pxTask, char
*pcTaskName)
#elif (defined(__GNUC__) || defined(__ti_version__))
void __attribute__((weak))
vApplicationStackOverflowHook(TaskHandle_t pxTask, char *pcTaskName)
#endif
{
    /* default to spin upon stack overflow */
    while (1) {
    }
}
#endif

```

blink_one_func.c

```
/*
*****

* This project provides a POSIX demo using a blinking LED.
*
* A POSIX thread controls the blinking LED.
*
* This file implements the code that is not demo specific.
*/

/* Standard includes */
#include <stdio.h>

/* POSIX header files */
#include <pthread.h>

/* RTOS header files */
#include <FreeRTOS.h>
#include <task.h>

/* TI includes for driver configuration */
#include "ti_msp_dl_config.h"

typedef struct {
    GPIO_Regs * gpio;
    int ioMuxPin;
    int dlGpioPin;
```

```

        float hz;
    } threadArgs;

void *LED(void *arg0)
{

    threadArgs *data = (threadArgs*) arg0;
    int ms = (float)(1/data->hz) * 1000;

    printf("IO Mux Pin: %d\n", data->ioMuxPin);
    printf("DL GPIO Pin: %d\n", data->dlGpioPin);

    DL_GPIO_enablePower(data->gpio);

    DL_GPIO_initDigitalOutput(data->ioMuxPin);

    DL_GPIO_initDigitalInput(data->ioMuxPin);

    DL_GPIO_enableOutput(data->gpio, data->dlGpioPin);

    DL_GPIO_clearPins(data->gpio, data->dlGpioPin);

    while(1)
    {
        DL_GPIO_setPins(data->gpio, data->dlGpioPin);
        if(ms < 1000)

```

```

        usleep(ms * 1000);
    else
        sleep(ms / 1000);
    DL_GPIO_clearPins(data->gpio, data->dlGpioPin);
    if(ms < 1000)
        usleep(ms * 1000);
    else
        sleep(ms / 1000);
}
}

```

```

/* Stack size in bytes */

```

```

#define THREADSTACKSIZE 1024

```

```

/* Set up the hardware ready to run this demo */

```

```

static void prvSetupHardware(void);

```

```

int main(void)

```

```

{
    pthread_t threadRED, threadGREEN;
    pthread_attr_t attrs;
    struct sched_param priParam;
    int retcRED, retcGREEN;

```

```

    /* Initialize the system locks */

```

```

#ifdef __ICCARM__

```

```

    __iar_Initlocks();

```

```
#endif
```

```
/* Prepare the hardware to run this demo. */
```

```
prvSetupHardware();
```

```
/* Initialize the attributes structure with default values */
```

```
pthread_attr_init(&attrs);
```

```
/* Set priority, detach state, and stack size attributes */
```

```
priParam.sched_priority = 1;
```

```
retcRED = pthread_attr_setschedparam(&attrs,  
&priParam);
```

```
retcRED |= pthread_attr_setdetachstate(&attrs,  
PTHREAD_CREATE_DETACHED);
```

```
retcRED |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);
```

```
retcGREEN = pthread_attr_setschedparam(&attrs,  
&priParam);
```

```
retcGREEN |= pthread_attr_setdetachstate(&attrs,  
PTHREAD_CREATE_DETACHED);
```

```
retcGREEN |= pthread_attr_setstacksize(&attrs, THREADSTACKSIZE);
```

```
if (retcRED != 0) {
```

```
    /* failed to set attributes */
```

```
    while (1) {
```

```
    }
```

```
}
```

```
if (retcGREEN != 0) {
```

```
    /* failed to set attributes */
```

```
    while (1) {
```

```
    }  
}
```

```
threadArgs redArgs;  
redArgs.gpio = GPIOA;  
redArgs.ioMuxPin = IOMUX_PINCM1;  
redArgs.dlGpioPin = DL_GPIO_PIN_0;  
redArgs.hz = 2;
```

```
threadArgs greenArgs;  
greenArgs.gpio = GPIOB;  
greenArgs.ioMuxPin = IOMUX_PINCM58;  
greenArgs.dlGpioPin = DL_GPIO_PIN_27;  
greenArgs.hz = 0.5;
```

```
retcRED = pthread_create(&threadRED, &attrs, LED, &redArgs);  
retcGREEN = pthread_create(&threadGREEN, &attrs, LED, &greenArgs);  
if (retcRED != 0) {  
    /* pthread_create() failed */  
    while (1) {  
    }  
}  
if (retcGREEN != 0) {  
    /* pthread_create() failed */  
    while (1) {  
    }  
}
```



```

    }

    /* Start the FreeRTOS scheduler */
    vTaskStartScheduler();

    return (0);
}

static void prvSetupHardware(void)
{
    SYSCFG_DL_init();
}

// #if (configSUPPORT_STATIC_ALLOCATION == 1)
// /*
//  * ===== vApplicationGetIdleTaskMemory =====
//  * When static allocation is enabled, the app must provide this
//  callback
//  * function for use by the Idle task.
//  */
// void vApplicationGetIdleTaskMemory(StaticTask_t
// **ppxIdleTaskTCBBuffer,
// StackType_t **ppxIdleTaskStackBuffer, uint32_t
// *pulIdleTaskStackSize)
// {
//     static StaticTask_t xIdleTaskTCB;
//     static StackType_t
// uxIdleTaskStack[configIDLE_TASK_STACK_DEPTH];

```

```

//      *ppxIdleTaskTCBBuffer    = &xIdleTaskTCB;
//      *ppxIdleTaskStackBuffer = uxIdleTaskStack;
//      *pulIdleTaskStackSize    = configIDLE_TASK_STACK_DEPTH;
//  }

// #if (configUSE_TIMERS == 1)
// /*
//  * ===== vApplicationGetTimerTaskMemory =====
//  * When static allocation is enabled, and timers are used, the app
//  must provide
//  * this callback function for use by the Timer Service task.
//  */
// void vApplicationGetTimerTaskMemory(StaticTask_t
**ppxTimerTaskTCBBuffer,
//      StackType_t **ppxTimerTaskStackBuffer, uint32_t
*pulTimerTaskStackSize)
// {
//      static StaticTask_t xTimerTaskTCB;
//      static StackType_t
uxTimerTaskStack[configTIMER_TASK_STACK_DEPTH];

//      *ppxTimerTaskTCBBuffer    = &xTimerTaskTCB;
//      *ppxTimerTaskStackBuffer = uxTimerTaskStack;
//      *pulTimerTaskStackSize    = configTIMER_TASK_STACK_DEPTH;
//  }
// #endif

```

```

// #endif

#if (configCHECK_FOR_STACK_OVERFLOW)
/*
 * ===== vApplicationStackOverflowHook =====
 * When stack overflow checking is enabled the application must
provide a
 * stack overflow hook function. This default hook function is
declared as
 * weak, and will be used by default, unless the application
specifically
 * provides its own hook function.
 */
#if defined(__IAR_SYSTEMS_ICC__)
__weak void vApplicationStackOverflowHook(
    TaskHandle_t pxTask, char *pcTaskName)
#elif (defined(__TI_COMPILER_VERSION__))
#pragma WEAK(vApplicationStackOverflowHook)
void vApplicationStackOverflowHook(TaskHandle_t pxTask, char
*pcTaskName)
#elif (defined(__GNUC__) || defined(__ti_version__))
void __attribute__((weak))
vApplicationStackOverflowHook(TaskHandle_t pxTask, char *pcTaskName)
#endif
{
    /* default to spin upon stack overflow */
    while (1) {
    }
}

```

```
}
```

```
#endif
```